T6 - Offline password cracking using John the Ripper (JtR)

The periodic verification of password quality is a fundamental step in all security architectures and it can be quite effective in limiting the risk of intrusions and data leaks. There are many different software tools for both defensive and offensive setups. It is worth noting that passwords are not used only for user authentication but are also part of other security mechanisms such as Wi-Fi network authentication, derivation of encryption keys, and so on.

In this tutorial, we will introduce the main features of the most famous tool for password cracking which is John the Ripper (JtR). The cracking efficiency of JtR depends on many factors such as the execution architecture that is available (e.g., CPU vs. GPU) but JtR is surely the most popular tool for this task and it is packed with useful features.

John the Ripper (JtR)

JtR is software released with an Open Source license and it is available in both free and paid (i.e., "pro") versions for several operating systems and hardware architectures.

```
> wget https://www.openwall.com/john/k/john-1.9.0-jumbo-1.tar.gz
> tar xvfz john-1.9.0-jumbo-1.tar.gz
> cd john-1.9.0-jumbo-1/src/
> ./configure
```

It is possible that this last command will return an error requiring the installation of some extra packages (e.g., tools or libraries). For example: "configure: error: JtR requires OpenSSL headers being installed". After installing the required packages, the configuration script must be executed again[1]. If now everything is fine, it is possible to proceed with the compilation of JtR.

[1] On some Linux distributions an error is generated when compiling with "gcc" version 11. An easy solution is to use the working version of JtR (i.e., community "jumbo" version) directly from the official git repository. The repository can be locally cloned using the following command: "git clone https://github.com/openwall/john.git" (obviously "git" must be available in the host computer). The following configuration and compilation steps do not change.

```
> make -s clean && make -sj4
```

If all the needed libraries and the compiling tools are properly installed then, in a few minutes, we will obtain the binaries of JtR. They will be in the directory: “john-1.9.0-jumbo-1/run”.

Cracking the passwords

Now that we have a working instance of JtR, we can verify the strength of the passwords that are used in a specific system.

In all modern UNIX-like systems (and derivatives) the information about the users (e.g., username and password) is split between two different files. The first file (i.e., /etc/passwd) contains the usernames and some ancillary information about users (e.g., the user identifier named UID). This file is readable by all unprivileged users in the system as well as by every process running on the system. The second file (i.e., /etc/shadow) contains the salted and hashed passwords and it is readable by users and processes with specific privileges (i.e., superuser). JtR requires a single input file with all the information described above. This can be easily done using a tool that is bundled with JtR.

```
> ./unshadow passwd-fake shadow-fake > passwd.1
```

Note: on Virtuale you can find two example files (e.g., passwd-fake and shadow-fake) to study this tutorial without the need to use usernames and passwords of a real system.

Everything is now ready to run JtR. It is a complex tool with many features. The simplest way to use it is the “**single crack**” mode.

```
> ./john --single passwd.1
```

Using this mode, JtR tries to guess the passwords using only the information that is available in the input file and adding only a limited amount of common variations. This is a very quick approach (in terms of execution time) that can guess only very weak passwords. For example, when the password is strongly related to the username.

A more complex but also more promising approach is the usage of a “**wordlist**” (i.e., a dictionary, a collection of common words).

```
> ./john  --wordlist=password.lst passwd.1
```

The wordlist used in this example is provided by JtR as part of the distribution package. It is also possible to use more complex and complete dictionaries. Another good idea would be to use a localized wordlist that considers the country and culture of attacked users. With this command, the execution will require a larger amount of time, which is strictly dependent on the size of the wordlist file.

The last approach, among the basic ones provided by JtR, is called "**incremental**" and it is based on an exhaustive search (i.e., brute-force) of all the possible passwords. Even a brute-force attack can be conducted smartly. For example, it is possible to choose the frequency of characters based on the language typically used by the attacked users (e.g., giving a zero probability to characters that are not on the keyboard) or to restrict the analysis to a specific range in the password length (e.g., according to the security policy of the attacked organization).

```
> ./john  --incremental passwd.1
```

It is quite obvious that, in most cases, **we cannot expect to finish the brute-force exploration in an acceptable amount of time**.

A bit more advanced mode of usage combines a "**wordlist**" with a set of rules for the "**variations**" of the words included in the list. In practice, each word is varied using a set of typical variations used by humans to generate passwords. For example, adding numbers or symbols before or after a dictionary word.

```
> ./john  --wordlist=password.lst --rules passwd.1
```

The runtime details about the execution can be found in the file "john.log" that is created in the working directory. When the execution is finished, all the recovered passwords can be found with:

```
> ./john passwd.1 -show
```

PDF cracking

JtR can be also used to verify the quality of passwords used to protect pdf documents (i.e., opening and modification). We start our analysis by creating two new documents, for example using LibreOffice (since it features the creation of "protected" pdf files). The two documents are protected using two different passwords:

1. **a+40!-**
2. **ThaliX_J_18/05/2008**

The two passwords are quite different both in structure and length. **What is your preferred one? In other words, which of them is more secure concerning password cracking?**

We can verify the quality of these passwords using JtR. JtR is unable to work directly on pdf files. Hence, the first step is to extract the hashed password that is contained in each pdf file.

The content of a "protected" pdf document is encrypted using a symmetric block cipher (e.g., 256-bit AES) but the password is also included (in hashed-form) inside the pdf file. The presence of the hashed password enables the usage of a password cracker as described in this part of the document. **Try to guess why the designers have decided to include also the hashed password.** In other words, is it really necessary to have the hashed password included in the pdf? What would be the consequences of not including the hashed password in the pdf?

For the sake of precision, the symmetric block cipher used to encrypt the pdf content needs an encryption key (i.e., binary and fixed length) and not a password (i.e., characters and user-chosen length). Therefore, also in this case a hash function needs to be used to perform a derivation from the user-chosen password to the encryption key that is suitable for the AES cipher. It is worth noting that this point is not strictly related to the question reported in bold above.

```
> ./pdf2john.pl <filename>.pdf > hash.txt
```

The obtained hash code can now be processed by JtR.

```
> ./john --format=pdf hash.txt
```

GPU-based cracking of passwords

GPUs are execution architectures that often permit a significant speedup in the execution of password cracking compared to CPUs. The structure of the problem is quite good for parallelization and the modern GPUs often implement (in hardware) the main encryption ciphers. JtR has limited GPU support, and more appropriate tools are available for password cracking on GPU. For example: Hashcat.